

Représentations vectorielles

Pourquoi représenter les textes en vecteurs ?

- Les ordinateurs ne comprennent pas les mots directement.
- Pour traiter du texte avec des algorithmes, on doit transformer les mots/phrases en nombres.
- Dans le but d'exploiter les données textuelles, les données peuvent être transformées en vecteurs auxquels s'appliquent les algorithmes.
- Une représentation vectorielle = une liste de nombres qui résume le contenu du texte.
-

Représentations vectorielles

- Représentations vectorielles classiques :
 - sac de mots, n-gram, TF-IDF
- Word embeddings
 - Word2Vec, GloVe, FastText

Sac de mots (*Bag of Words*, BoW)

- Un document (ou un texte donné) est représenté par les vecteurs de fréquence des mots qui le composent, permettant **pour chaque mot** de voir **sa fréquence** au sein de l'ensemble de données.
- Un texte est transformé en comptant les mots qui apparaissent **sans tenir compte de l'ordre**.
- L'**ensemble de données** est ainsi **représenté sous forme d'une matrice** utilisée comme entrée de l'algorithme et **composée de l'ensemble des N phrases ou documents, chacun représenté par un vecteur de dimension égale à la taille du vocabulaire**.

Sac de mots (*Bag of Words*, BoW)

Texte 1 : *Le chat dort sur le canapé.*

Texte 2 : *Le chien dort sur le tapis.*

Vocabulaire (ensemble des mots uniques) = {chat, chien, dort, sur, le, canapé, tapis}

Vecteur du texte 1 = [1, 0, 1, 1, 2, 1, 0]

Vecteur du texte 2 = [0, 1, 1, 1, 2, 0, 1]

Sac de mots (*Bag of Words*, BoW)

- **Avantages :**

- Simple, facile à calculer.

- **Limites :**

- Perte de l'ordre des mots car une phrase est considérée comme un **ensemble de mots non ordonnés dont la position n'est pas prise en compte**

chien mord l'homme = homme mord le chien

- Les mots fréquents (*le, sur*) dominant les vecteurs.

N-gram

- comme le sac de mots, mais on prend en compte des **séquences de n mots** (bi-gram, tri-gram...)
- *cadre très agréable et service prévenant*
 - *cadre très* = un bi-gramme
 - *cadre très agréable* = un tri-gramme
- **retiennent la structure séquentielle des données d'origines** et par conséquent, l'application des n-grammes dans la représentation **rend les données plus informatives.**
- le **désavantage** de ce modèle
 - plus n est élevé, plus la représentation aboutit à **un vecteur de grande dimension**
 - les **n-grammes ayant une valeur de n élevée** ne contribuent pas toujours à une amélioration de la performance.

N-gram

Document 1 : *Ce design est super intéressant.*

Document 2 : *Je trouve ce design chouette.*

Document 3 : *Ce design est moche.*

	ce design	design est	est super	super intéressant	je trouve	trouve ce	ce design	design chouette
Doc 1	1	1	1	1	0	0	0	0
Doc 2	1	0	0	0	1	1	1	0
Doc 3	1	1	0	0	0	0	0	1

TF-IDF (*Term Frequency-Inverse Document Frequency*)

Pour corriger le problème des mots trop fréquents, on pondère les mots.

- Le TF-IDF (Term Frequency – Inverse Document Frequency) est une mesure statistique qui
 - convertit des textes en vecteurs numériques
 - reflète **l'importance d'un terme pour un document** dans une collection de documents
 - **pondère l'importance de chaque mot dans un document par rapport à un ensemble de documents**
 - **donne un poids élevé aux mots fréquents dans un document mais rares dans le corpus.**
 - **Le poids de chaque mot y est calculé en divisant la fréquence du mot par le nombre de document contenant le mot donné.**

.

TF-IDF (*Term Frequency – Inverse Document Frequency*)

- **TF (Term Frequency) :**
 - fréquence d'un mot dans le document.
- **IDF (Inverse Document Frequency) :**
 - mesure **l'importance d'un mot dans l'ensemble des documents.**
 - plus un mot est fréquent dans *tous* les documents, moins il est important.
le apparaît dans tous les documents => faible poids.
chat est spécifique à un document => poids plus élevé.

Les vecteurs mettent en avant les mots distinctifs de chaque document.

TF-IDF (*Term Frequency-Inverse Document Frequency*)

Document 1 : *Ce design est super intéressant.*

Document 2 : *Je trouve ce design chouette.*

Document 3 : *Ce design est moche.*

	Doc 1	Doc 2	Doc 3
design	0.345205	0.385372	0.425441
être	0.444514	0.000000	0.547832
super	0.584483	0.000000	0.000000
intéressant	0.584483	0.000000	0.000000
trouver	0.000000	0.652491	0.000000
chouette	0.000000	0.652491	0.000000
moche	0.000000	0.000000	0.720333

	Avantages	Inconvénients
Sac de mots (<i>Bag of Words</i>)	• Très simple à comprendre et à mettre en œuvre • Rapide à calculer.	• Perd totalement l'ordre des mots. • Vecteurs très grands si le vocabulaire est riche. • Sensible aux mots fréquents non informatifs (<i>le, et, de</i>).
N-gram	• Conserve un peu d'information sur l'ordre et le contexte local • Peut capturer des expressions utiles ("machine learning", "New York")	• Explosion de la taille du vocabulaire (plus n est grand, plus il y a de combinaisons possibles). • Données clairsemées (beaucoup de n-grammes rares ou absents)
Tf-Idf	• Réduit l'importance des mots très fréquents • Met en valeur les termes caractéristiques d'un document. • Améliore souvent les performances par rapport au simple sac de mots.	• Perd toujours l'ordre des mots • Dépend de la taille du corpus (un mot peut paraître rare dans un petit corpus mais fréquent dans un grand)

Word embeddings

- Word2Vec,
- GloVe
- FastText

Pourquoi les embeddings ?

- Avec sac de mots / TF-IDF, chaque mot est une case indépendante → pas de notion de similarité entre les mots.
chat/félin n'ont rien à voir dans un sac de mots, alors qu'ils sont proches en sens.
- Les **word embeddings** représentent les mots par des **vecteurs de petite dimension** (ex. 100, 300) qui **capturent** leur **sens** et **relations sémantiques**.

Les plongements lexicaux (words embedding)

- Les **plongements lexicaux** sont basés sur l'**hypothèse distributionnelle** (Harris 1951; Firth 1957) qui suppose que **des mots qui apparaissent dans des contextes similaires ont des sens proches**.
- Deux **mots ayant une signification proche** ont une représentation similaire, autrement dit, les mots **sont projetés à proximité dans l'espace vectoriel**.
- Les plongements lexicaux
 - **cherchent à associer à chaque mot un vecteur qui encode le contexte d'apparition de ce mot**.
 - visent à créer une **représentation vectorielle du vocabulaire capturant leur signification et prennent en compte une certaine similarité sémantique** entre des mots.

Modèles sémantiques vectoriels

- un mot est représenté par un vecteur, c.-à-d. « une liste de nombres **reflétant les statistiques de cooccurrences du mot dans un corpus** donné » (Miletic, 2022, p. 126)
- Les modèles vectoriels nous permettent de **calculer la similarité entre deux vecteurs** et donc entre deux mots.
- Il est possible d'utiliser différentes distances comme **le cosinus** ou **la distance euclidienne**.
- Les mots, les phrases ou même les documents peuvent être représentés par des vecteurs numériques dans **un espace sémantique multidimensionnel**.

Modèles sémantiques vectoriels

- Deux types de modèles basés sur :
 - les **fréquences des co-occurrences dans un corpus**
 - proposent des **vecteurs avec un grand nombre de dimensions** (quelques dizaines de milliers) **correspondant à la taille du vocabulaire** et un grand nombre de valeurs nulles (matrices creuses) (Jurafsky et Martin, 2025).
 - des **réseaux de neurones**
 - proposent des ***embeddings* denses avec quelques centaines de dimensions**
 - **la qualité de ces *embeddings* dépend directement de la qualité des données sur lesquelles ces modèles sont entraînés** (c.-à-d. dont sont extraits les contextes d'apparition des mots)

Word2Vec

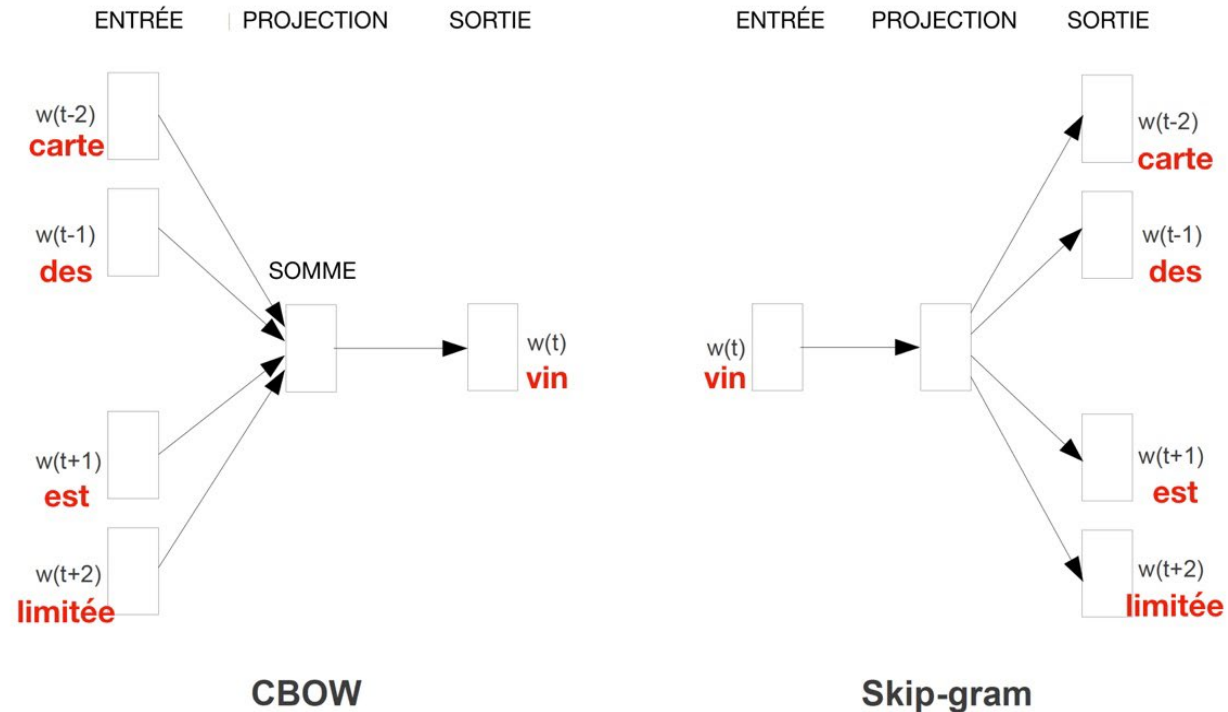
- entraîné sur un grand corpus en prédisant les mots voisins.
- **Skip-gram** :
 - prédit le contexte à partir d'un mot.
- **CBOW (Continuous Bag of Words)** :
 - prédit un mot à partir de son contexte.

Représentation vectorielle avec Word2vec

Seul petit bémol, la carte des vins est limitée et un peu chère.

CBOW : le mot cible *vins* est prédit à partir des mots constituant son contexte : *carte, des, est, limitée* en entrée.

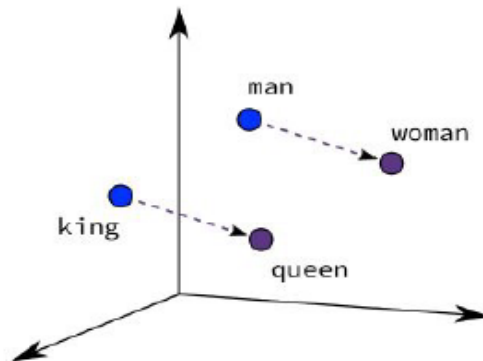
Skip-gram : l'inverse



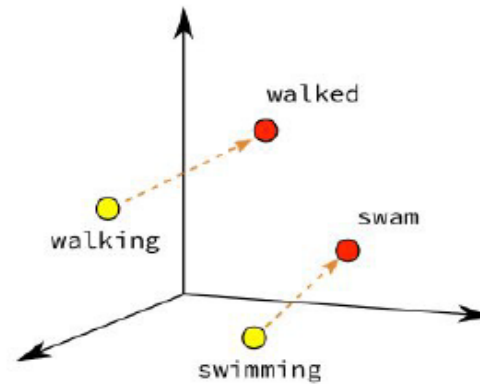
Projections des plongements de mots Word2Vec dans un espace deux-dimensionnel

Word2Vec encode des **propriétés sémantiques** et **syntaxiques**. Les vecteurs Word2Vec capturent les **régularités dans les contextes d'usage des mots**. => on peut observer des analogies :

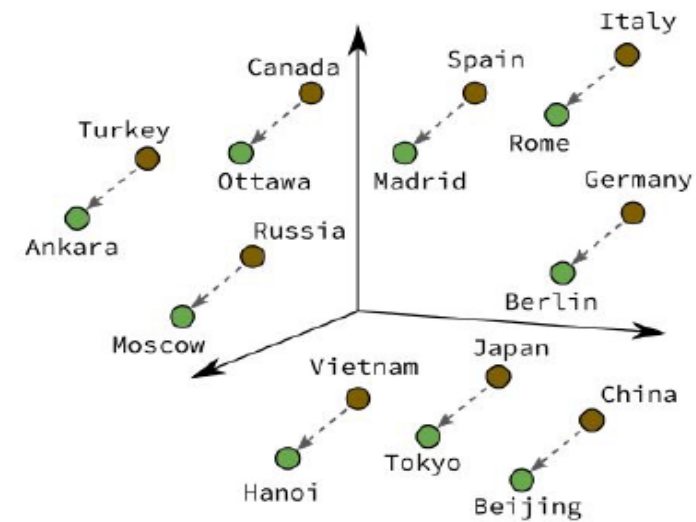
- le **vecteur de *king*** : Lorsqu'on lui ajoute des informations relatives à la "féminité" (le **vecteur pour "woman"**), on **obtient la représentation de *queen*** : $\text{vect}(\text{king}) - \text{vect}(\text{man}) + \text{vect}(\text{woman}) \approx \text{vect}(\text{queen})$
- **Paris - France + Italy = Rome**



Male-Female



Verb Tense



Country-Capital

Représentation vectorielle avec Word2vec

- **Il existe des modèles Word2vec, déjà entraînés sur des données massives, qui sont disponibles en ligne, notamment pour l'anglais.**
- Grâce aux modèles pré-entraînés, il n'est pas nécessaire de chercher un corpus de grande taille d'une part, et d'autre part, le temps d'entraînement est gagné.
- Ex. deux modèles Word2vec (CBOW et skip-gram entraînés sur le corpus frWiki (Wikipédia) par Fauconnier (2016)

Word2Vec

- Avantages :
 - Capte les similarités sémantiques
 - Compact (vecteurs courts)
 - Rapide à entraîner
- Inconvénients
 - **Ne gère pas les mots inconnus**
 - Un seul vecteur par mot, même si plusieurs sens => **ne gère pas la polysémie**

GloVe (Global Vectors)

- basé sur les **statistiques globales de cooccurrence de mots** dans un corpus.
- **Analyse la probabilité qu'un mot apparaisse près d'un autre**, et apprend des vecteurs qui respectent ces cooccurrences.

GloVe (Global Vectors)

- Avantages :
 - **Combine** les points forts **des modèles basés sur la prédiction** (Word2Vec) et **les statistiques globales** (matrices de cooccurrence).
 - Bons résultats sur des **relations sémantiques et analogies**.
- Inconvénients
 - Même limitation qu'avec Word2Vec : un vecteur par mot (**pas de polysémie**)
 - Nécessite un gros corpus pour être efficace.

Word2Vec vs GloVe

Word2Vec

- Approche **prédictive** : L'idée est d'**entraîner un petit réseau de neurones** pour prédire le contexte d'un mot (Skip-gram) ou prédire un mot à partir de son contexte (CBOW).
- On ne stocke pas explicitement les **cooccurrences** : elles sont **appries indirectement par l'entraînement**
- C'est comme si le modèle "jouait à deviner les mots manquants" et apprenait des vecteurs en conséquence.
- **apprentissage local** basé sur des **fenêtres de contexte**.

GloVe

- **Approche statistique** : construit une **matrice de cooccurrence globale** (combien de fois un mot apparaît près d'un autre, dans tout le corpus) puis **il entraîne les vecteurs** pour que leurs produits scalaires correspondent à ces statistiques de cooccurrence.
- utilise une **information globale** (statistiques sur tout le corpus), **pas seulement des contextes locaux**.
- **apprentissage global** basé sur des **cooccurrences**

Word2Vec vs Glove

Commun

- Word2Vec et GloVe **utilisent** tous les deux **des corpus pour apprendre des vecteurs de mots**

Différence

- Word2Vec : **prédictif**, basé sur des **échantillons de contexte** (fenêtres locales).
- GloVe : **statistique**, basé sur une **matrice globale de cooccurrences**.

FastText

- amélioration de Word2Vec => décompose les mots en sous-unités (caractères, n-grammes)
- même un mot inconnu peut être représenté à partir de ses morceaux.
apprentissage => appr, entis, sage

FastText

- FastText ([Joulin et al., 2016](#)) est une **extension de Word2vec**
- fondé sur Skip-gram mais
 - **chaque mot est représenté comme un ensemble de n-grammes de caractères** comme une somme de n-grammes de caractères
 - **FastText prend en compte les informations relatives aux sous-mots**
 - FastText peut donc générer un vecteur pour **un mot hors vocabulaire**, car il le décompose en sous-unités
- **propose un unique vecteur associé à un mot** du vocabulaire.
- toutes les occurrences de ce mot sont traitées de la même façon
=> **un mot polysémique est représenté par un unique vecteur** qui agrège ses différents sens.

FastText

- Des **symboles spéciaux** < et > sont **ajoutés au début et à la fin de chaque mot** permettant de distinguer les préfixes des suffixes des autres séquences de caractères.
- **Le mot est également inclus dans l'ensemble de ces n-grammes** afin d'apprendre une représentation de chaque mot en plus des n-grammes de caractères.
- Ex : le mot *nocturne* où $n=3$ sera représenté par les n-grammes de caractères :
 <no, noc, oct, ctu, tur, urn, rne, ne > et <nocturne>.
- FastText permet ainsi de partager les représentations entre les mots, ce qui permet d'apprendre des représentations fiables pour les **mots rares**.
- Il a été **entraîné sur des données de Wikipedia en 157 langues** : arabe, tchèque, allemand, anglais, espagnol, français, italien, roumain, russe, etc.

FastText

- **Avantages :**

- **Gère mieux les mots rares et inconnus**
- **Capture la morphologie**
- **Plus robuste** que Word2Vec et GloVe **pour de petits corpus**

- **Inconvénients**

- **Même limitation : un vecteur par mot (pas de polysémie, pas de désambiguïsation complète)**
- **Plus coûteux en calcul**

Les embeddings statiques: la conclusion

- **Word2Vec :**
 - apprend par contexte
 - vecteurs de mots avec relations sémantiques.
- **GloVe :**
 - basé sur cooccurrence globale
 - vecteurs cohérents au niveau corpus.
- **FastText**
 - ajoute les sous-mots
 - robuste pour mots rares / morphologie.

Représentations contextualisées des mots

- Contrairement aux modèles statiques, les contextuels **calculent un vecteur de mot en prenant en compte le contexte d'apparition**, c'est-à-dire qu'ils prennent en compte les autres mots et leur position relative.
- Chaque mot est projeté dans le texte avec une représentation propre à son contexte, ce qui permet de capturer des nuances de sens même pour des mots identiques apparaissant dans des phrases différentes.
- Elles permettent à un modèle de générer **un vecteur différent pour chaque occurrence d'un mot, selon le contexte** dans lequel il apparaît.

BERT

- BERT (*Bidirectional Encoder Representations from Transformers*), développé par Google en 2018.
- Dans les représentations statiques (Word2Vec, GloVe), chaque mot a un vecteur fixe quelle que soit la phrase,
- Les modèles comme BERT produisent **un vecteur dynamique**, qui change selon la phrase :

Je me suis assis sur un banc.

Un banc de poissons nageait près du rivage.

avec un embedding contextuel (BERT), le mot "banc" aura deux vecteurs différents, car son contexte n'est pas le même.

banc = chaise dans « *Je me suis assis sur un banc* »,

banc = poissons dans « *Un banc de poissons nageait près du rivage* »